

K8s - Partie n°1

> **Étudiant :** David MELOCCO, TD1 BUT3 **Date :** vendredi 6 novembre 2026

Prérequis

L'ensemble des travaux pratiques sont faits sous **Windows** et **PowerShell**. Une configuration **WSL2** est déjà installée pour **Docker Desktop**. L'extension **VS Code Kubernetes**, proposée par **Microsoft**, a été installée.

> 📦 Le dépôt **GitHub** est open-source, disponible ici : https://github.com/ThFoxY/kubernetes-e-learn-lab

minikube

```
minikube start --driver=docker

😄  minikube v1.39.0 sur Microsoft Windows 11 Home 25H2
✨  Utilisation du pilote docker basé sur la configuration de l'utilisateur
📌  Utilisation du pilote Docker Desktop avec le privilège root
👍  Démarrage du nœud "minikube" primary control-plane dans le cluster "minikube"
🚜  Extraction de l'image de base v0.0.51...
```

```
💾  Téléchargement du préchargement de Kubernetes v1.37.
0...
    > preloaded-images-k8s-v18-v1...:  347.58 MiB / 347.58
MiB  100.00% 947.72
    > gcr.io/k8s-minikube/kicbase:  507.61 MiB / 507.61 MiB
100.00% 1.17 MiB p
🔥  Création de docker container (CPU=2, Memory=3900Mo) ...
📦  Préparation de Kubernetes v1.37.0 sur containerd 2.3.
4...
🔗  Configuration de CNI (Container Networking Interfac
e)...
🔎  Vérification des composants Kubernetes...
    ▪ Utilisation de l'image gcr.io/k8s-minikube/storage-pr
ovisioner:v5
🌟  Modules activés: storage-provisioner, default-storagecl
ass
💡  kubectl introuvable. Si vous en avez besoin, essayez :
'minikube kubectl -- get pods -A'
🏄  Terminé ! kubectl est maintenant configuré pour utilise
r "minikube" cluster et espace de noms "default" par défau
t.

minikube status

minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

kubectl

```
kubectl version

Client Version: v1.36.1
```

```
Kustomize Version: v5.8.1
Server Version: v1.37.0
```

Namespaces

1. Créer les 3 namespaces (fichiers YAML et non depuis kubectl) Documentation

Le répertoire racine s'appelle `k8s-partie-1` et un dépôt Git y a été initialisé. La documentation officielle a été lue, et n'utilise pas `kubectl` pour créer les fichiers **YAML**.

`/front/namespace.yml`

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: front
  labels:
    name: front
```

`/database/namespace.yml`

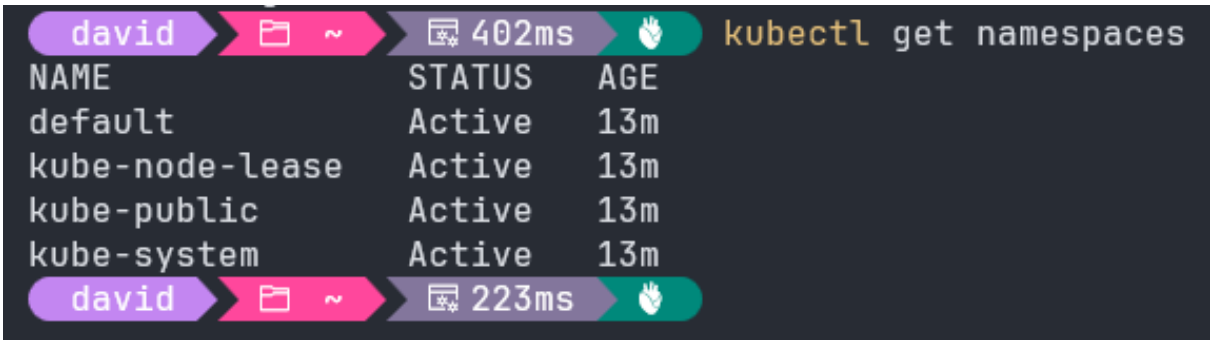
```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: database
  labels:
    name: database
```

`/middle/namespace.yml`

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: middle
```

```
  labels:
    name: middle
```

2. 📸 Visualiser la liste des namespaces sur le cluster Minikube



> La commande `kubectl get ns` est un raccourci.

3. Déployer les 3 namespaces

Pour chaque sous-dossier (`front` , `database` , `middle`) :

```
kubectl apply -f .\namespace.yml
namespace/front created

kubectl apply -f .\namespace.yml
namespace/database created

kubectl apply -f .\namespace.yml
namespace/middle created
```

4. Visualiser la configuration d'un namespace créé

```
kubectl describe namespace front

Name:         front
Labels:       kubernetes.io/metadata.name=front
              name=front
Annotations:  <none>
Status:       Active
```

```
No resource quota.

No LimitRange resource.

kubectl describe namespace database

Name:         database
Labels:       kubernetes.io/metadata.name=database
              name=database
Annotations:  <none>
Status:       Active

No resource quota.

No LimitRange resource.

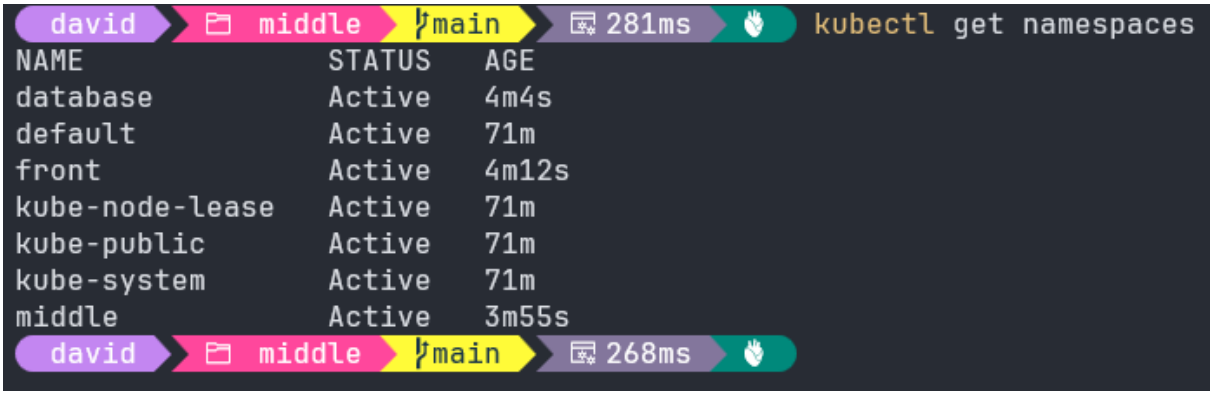
kubectl describe namespace middle

Name:         middle
Labels:       kubernetes.io/metadata.name=middle
              name=middle
Annotations:  <none>
Status:       Active

No resource quota.

No LimitRange resource.
```

5. 📸 Visualiser la liste des namespaces sur le cluster Minikube



Nginx

1. 📄 Créer un fichier YAML `nginx-deployment.yml`, au sein de ce fichier ajoutez y une ressource de type `Deployment` qui utilise le nom `nginx`. Cette ressource match et déploie `2` replicas d'un pod qui a pour label `app: reverse-proxy`. Ce pod contient un conteneur docker nommé `nginx` qui utilise l'image publique `nginx` dans sa version `1.27.1-alpine`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: reverse-proxy
  template:
    metadata:
      labels:
        app: reverse-proxy
    spec:
      containers:
                - name: nginx
                image: nginx:1.27.1-alpine
                ports:
                - containerPort: 80
```

> **Les commentaires questions / réponses ont été retirés pour la clarté. Le nom est `nginx-deployment` afin d'éviter la confusion future avec le Service du même nom.**

La version **1.27.1-alpine** date d'il y a deux ans (https://hub.docker.com/_/nginx/tags?name=1.27.1-alpine), mais je l'ai gardé pour le cadre de cet exercice.

L'extension VS Code Kubernetes indique une mauvaise pratique de code : il faut inclure des `requests` et `limits` dans `resources` ("One or more containers do not have resources - this can cause noisy neighbor issues").

---

2. 📄 Créer un fichier YAML `nginx-service.yml`, au sein de ce fichier ajoutez- y une ressource de type `Service` qui a pour nom `nginx` Ce service doit utiliser le protocole `TCP` et écoute sur le port `8383`. Il redirige le flux sur le port `80` les pods quit on un label `app: reverse-proxy`

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  selector:
    app: reverse-proxy
  ports:
    - protocol: TCP
      port: 8383
      targetPort: 80
```

> **Le nom est `nginx-svc` afin d'éviter la confusion avec le Deployment du même nom.**

---

3. Déployer les ressources créées dans le namespace `front` Documentation

```
kubectl apply -f .\front\ --namespace=front
```

```
namespace/front unchanged
deployment.apps/nginx-deployment created
service/nginx-svc created
```

---

4. Vérifier les events du namespace et le bon démarrage des pods et de leurs conteneurs ❓ Quelles sont les 2 commandes utilisées ?

```
kubectl get events -n front

LAST SEEN   TYPE     REASON              OBJECT
MESSAGE
2m27s       Normal   Scheduled           pod/nginx-deployme
nt-5d68b47d97-s8p4r    Successfully assigned front/nginx-de
ployment-5d68b47d97-s8p4r to minikube
2m25s       Normal   Pulling             pod/nginx-deployme
nt-5d68b47d97-s8p4r    Pulling image "nginx:1.27.1-alpine"
2m16s       Normal   Pulled              pod/nginx-deployme
nt-5d68b47d97-s8p4r    Successfully pulled image "nginx:1.2
7.1-alpine" in 8.578s (8.579s including waiting). Image siz
e: 18593388 bytes.
2m16s       Normal   Created             pod/nginx-deployme
nt-5d68b47d97-s8p4r    Container created
2m16s       Normal   Started             pod/nginx-deployme
nt-5d68b47d97-s8p4r    Container started
2m27s       Normal   Scheduled           pod/nginx-deployme
nt-5d68b47d97-vs5bp    Successfully assigned front/nginx-de
ployment-5d68b47d97-vs5bp to minikube
2m25s       Normal   Pulling             pod/nginx-deployme
nt-5d68b47d97-vs5bp    Pulling image "nginx:1.27.1-alpine"
2m15s       Normal   Pulled              pod/nginx-deployme
nt-5d68b47d97-vs5bp    Successfully pulled image "nginx:1.2
7.1-alpine" in 1.101s (9.613s including waiting). Image siz
e: 18593388 bytes.
2m15s       Normal   Created             pod/nginx-deployme
nt-5d68b47d97-vs5bp    Container created
2m15s       Normal   Started             pod/nginx-deployme
```

```
nt-5d68b47d97-vs5bp    Container started
2m27s       Normal   SuccessfulCreate    replicaset/nginx-d
eployment-5d68b47d97   Created pod: nginx-deployment-5d68b4
7d97-s8p4r
2m27s       Normal   SuccessfulCreate    replicaset/nginx-d
eployment-5d68b47d97   Created pod: nginx-deployment-5d68b4
7d97-vs5bp
2m27s       Normal   ScalingReplicaSet   deployment/nginx-d
eployment              Scaled up replica set nginx-deployme
nt-5d68b47d97 from 0 to 2
```

`--field-selector type=Warning` en option permettra de filtrer uniquement les **warnings**.

```
kubectl get pods -n front

NAME                                READY   STATUS    RESTA
RTS   AGE
nginx-deployment-5d68b47d97-s8p4r   1/1     Running   0
3m39s
nginx-deployment-5d68b47d97-vs5bp   1/1     Running   0
3m39s
```

---

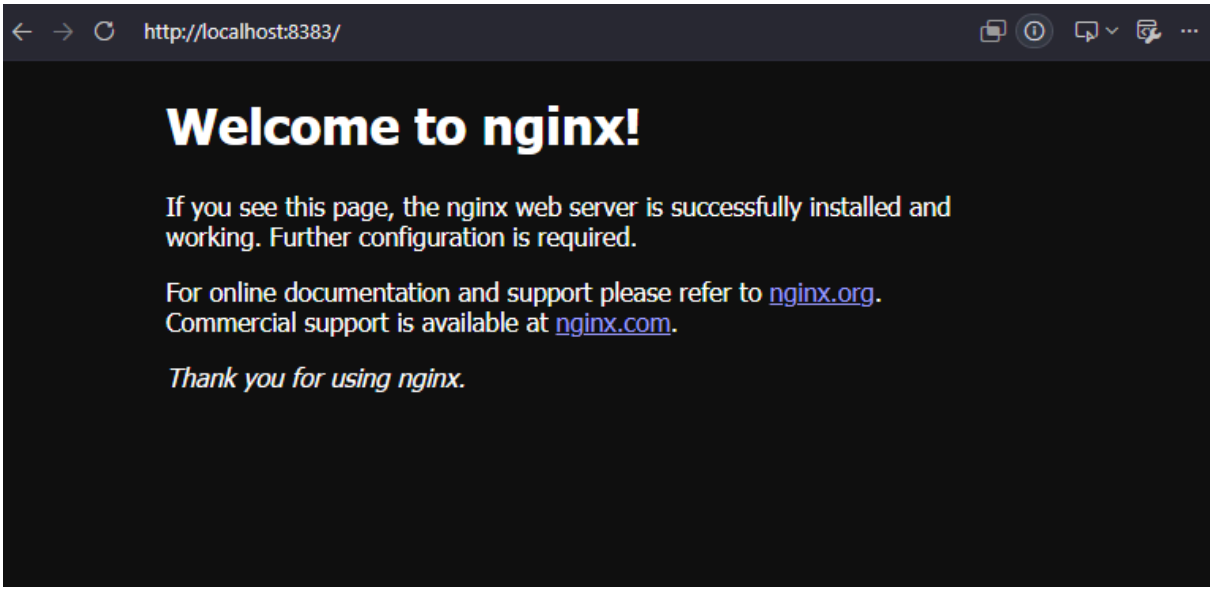
5. Créer une redirection de port (port-forwarding) pour contacter votre service qui écoute sur le port 8383. Si la configuration et le déploiement des ressources sont corrects, le serveur web Nginx affiche sa page par défaut.

```
kubectl port-forward -n front svc/nginx-svc 8383:8383

Forwarding from 127.0.0.1:8383 -> 80
Forwarding from [::1]:8383 -> 80
```

---

6. Accéder depuis votre navigateur : http://localhost:8080



---

7. ❓ A quoi correspond les pointillés violet ?

L'encadré en pointillés violet correspond aux **Worker Nodes**.

8. ❓ A quoi correspond les pointillés ?

L'encadré en pointillés correspond aux **Cluster K8s**.